Git & GitHub

Version Control and Collaboration from the Command Line

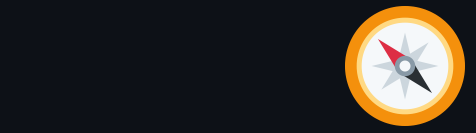
🔐 IT 360 | 🌿 Branches | 🤝 Collaboration

Topics

- **What** Git and GitHub are — and **why** we use them
- Setting up Git and the **basic commands**
- **Branches** — creating, switching, merging
- **Collaboration** on GitHub from the command line
- Merge **conflicts** and good team habits
- Git as **digital evidence**

🎯 Goals

- Explain the difference between **Git** and **GitHub**
- Create a repository and record changes with `add` / `commit`
- Inspect history with `status`, `log`, and `diff`
- Use **branches** to isolate work and **merge** it back
- Clone, push, pull, and open a **pull request** with a team



What & Why

What Is Version Control?

- A system that **records changes** to files over time
- Lets you **recall** any earlier version, see **who** changed **what** and **when**
- Replaces the "final_v2_REALLY_final.docx" approach

WITHOUT IT

Copies of folders, emailed zip files, overwritten work, no history.

WITH IT

One project, full history, safe experiments, easy rollback.

Git vs. GitHub

GIT — THE TOOL

Free, open-source **distributed version control** system created by Linus Torvalds (2005) for the Linux kernel. Runs **locally** on your machine.

GITHUB — THE SERVICE

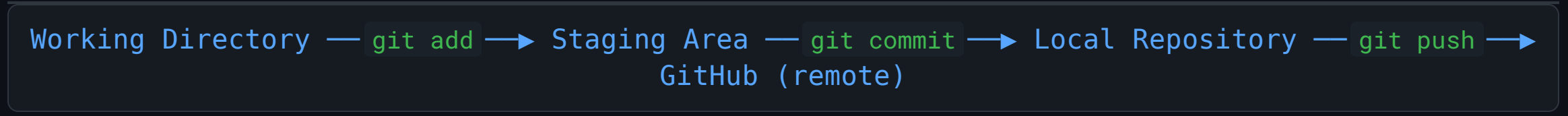
A **hosting platform** for Git repositories. Adds a web UI, pull requests, issues, Actions (CI/CD), wikis, and access control.

> **Git works without GitHub.** GitHub is one of several hosts (also GitLab, Bitbucket, Gitea).

Why Use Git & GitHub?

Benefit	What it gives you
History	Every change is saved — undo mistakes, see who did what
Branching	Try new ideas without breaking working code
Distributed	Every clone is a full backup of the project
Collaboration	Many people work on the same project in parallel
Code review	Pull requests let teammates review before merging
Portfolio	Public repos show employers your work
Industry standard	Used by almost every software & security team

How Git Thinks: Three Areas



WORKING DIRECTORY

The files you are editing right now.

STAGING AREA

Changes you have **chosen** to include in the next commit.

REPOSITORY (.GIT)

The saved **snapshots** (commits) and their history.

Key Vocabulary

Term	Meaning
Repository (repo)	A project folder tracked by Git (has a hidden `.git/` directory)
Commit	A saved snapshot with a message, author, timestamp, and unique **hash**
Branch	A movable pointer to a line of commits
HEAD	Pointer to the commit/branch you currently have checked out
Remote	A copy of the repo hosted elsewhere (e.g., `origin` on GitHub)
Clone	A full local copy of a remote repo
Pull request (PR)	A GitHub request to merge one branch into another

⌨ Basic Commands

One-Time Setup

```
# Check that Git is installed
git --version

# Identify yourself — stored in every commit
git config --global user.name  "Your Name"
git config --global user.email "you@ilstu.edu"

# Use "main" as the default branch name
git config --global init.defaultBranch main

# Review your settings
git config --list
```

> Use the **same email** as your GitHub account so commits link to your profile.

Authenticating to GitHub

GitHub no longer accepts account passwords for Git operations. Pick one:

SSH KEY (RECOMMENDED)

`ssh-keygen -t ed25519 -C "you@ilstu.edu"`

Paste `~/.ssh/id_ed25519.pub` into GitHub → Settings → SSH keys.

Test: `ssh -T git@github.com`

HTTPS + TOKEN / GH CLI

Create a **Personal Access Token**, or run `gh auth login` to have the GitHub CLI handle credentials.

Starting a Repository

New project

```
mkdir my-project
cd my-project
git init
```

Creates the hidden `.git/` folder.

Existing project on GitHub

```
git clone git@github.com:user/repo.git
cd repo
```

Downloads the **full history** and sets up `origin`.

The Everyday Loop

```
git status                  # What changed? What is staged?
git add notes.md            # Stage one file
git add .                   # Stage everything in this folder
git commit -m "Add lab 3 notes"   # Save a snapshot
git push                    # Send commits to GitHub
git pull                    # Get teammates' commits
```

edit → `git add` → `git commit` → `git push` → repeat

> **Commit early, commit often.** Small commits are easier to review and undo.

Inspecting History & Changes

Command	Shows
`git status`	Modified, staged, and untracked files
`git diff`	Unstaged changes, line by line
`git diff --staged`	Changes that will go into the next commit
`git log`	Full commit history
`git log --oneline --graph --all`	Compact history with branch graph
`git show <hash>`	Details of one commit
`git blame <file>`	Who last changed each line

Undoing Things

Situation	Command
Discard edits to a file	`git restore file.txt`
Unstage a file	`git restore --staged file.txt`
Fix the **last** commit message / add a forgotten file	`git commit --amend`
Undo a commit **safely** (new commit that reverses it)	`git revert <hash>`
Move branch back, keep changes	`git reset --soft HEAD~1`

> ⚠ `git reset --hard` and rewriting **pushed** history destroy work — avoid on shared branches.

Ignoring Files: `.gitignore`

Tell Git which files should **never** be tracked:

```
# Dependencies & build output
node_modules/
__pycache__/
*.log

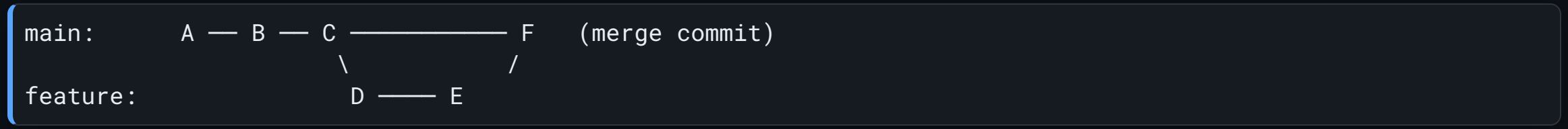
# Secrets — NEVER commit these
.env
*.pem
id_rsa*
```

- Add `.gitignore` **before** the first commit
- Templates: github.com/github/gitignore

🌿 Branches

What Is a Branch?

- An independent **line of development**
- Lightweight — just a pointer to a commit
- `main` holds stable work; feature branches hold work-in-progress



> Why? Experiment safely, work on several tasks at once, and keep `main` always working.

Branch Commands

Command	Action
`git branch`	List local branches (`*` = current)
`git branch -a`	List local **and** remote branches
`git switch -c feature/login`	Create a branch and switch to it
`git switch main`	Switch to an existing branch
`git branch -d feature/login`	Delete a merged branch
`git push -u origin feature/login`	Publish branch to GitHub & track it

> Older syntax you will still see: `git checkout -b <name>` = `git switch -c <name>`.

Merging a Branch

```
git switch -c feature/report     # 1. Branch off main
# ...edit files...
git add .
git commit -m "Draft findings section"   # 2. Commit on the branch

git switch main                  # 3. Go back to main
git pull                         # 4. Make sure main is current
git merge feature/report         # 5. Bring the work in
git branch -d feature/report     # 6. Clean up
```

FAST-FORWARD MERGE

main had no new commits — Git just moves the pointer forward.

THREE-WAY MERGE

Both branches changed — Git creates a **merge commit** joining them.

Branch Naming & Strategy

- Use descriptive, prefixed names:
  - `feature/add-scanner` · `fix/typo-readme` · `docs/lab-4`
- **One branch per task** — keep them short-lived
- Never commit directly to `main` on a team project
- Common model: **GitHub Flow**

```
branch from main ⟶ commit ⟶ push ⟶ open PR ⟶ review ⟶ merge ⟶ delete branch
```

🤝 Collaboration on GitHub

Remotes

A **remote** is a named URL for another copy of the repo.

```
git remote -v                                  # List remotes
git remote add origin git@github.com:user/repo.git   # Link a local repo
git push -u origin main                        # First push; sets upstream
```

Command	Direction	What it does
`git fetch`	⬇️	Download new commits, **don't** change your files
`git pull`	⬇️	`fetch` + `merge` into current branch
`git push`	⬆️	Upload your commits to the remote

Two Ways to Collaborate

SHARED REPOSITORY

Teammates are added as **collaborators** (Settings → Collaborators). Everyone clones the same repo and pushes branches to it.

Best for: class teams, small groups.

FORK & PULL

You **fork** (copy) someone's repo to your account, push to your fork, then open a PR back to the original (`upstream`).

Best for: open-source contributions.

Team Workflow — Command Line

```
# 1. Get the project (once)
git clone git@github.com:team/project.git && cd project

# 2. Start each task from an up-to-date main
git switch main
git pull

# 3. Create a branch for your task
git switch -c feature/nmap-section

# 4. Work, then commit
git add .
git commit -m "Add Nmap scan results section"

# 5. Publish the branch
git push -u origin feature/nmap-section

# 6. Open a pull request (web UI or GitHub CLI)
gh pr create --base main --title "Add Nmap section" --body "Closes #4"
```

Pull Requests

- A PR asks to **merge** your branch into `main`
- Teammates **review**: comment on lines, request changes, approve
- Push more commits to the same branch → the PR **updates automatically**
- Merge when approved; delete the branch

GitHub CLI	Action
`gh pr create`	Open a PR from current branch
`gh pr list`	List open PRs
`gh pr checkout 12`	Check out PR #12 locally to test it
`gh pr review 12 --approve`	Approve a PR
`gh pr merge 12 --squash --delete-branch`	Merge and clean up

Fork Workflow — Command Line

```
# Fork on GitHub (or: gh repo fork owner/project --clone)
git clone git@github.com:YOU/project.git && cd project

# Track the original repo as "upstream"
git remote add upstream git@github.com:owner/project.git

# Keep your fork in sync
git fetch upstream
git switch main
git merge upstream/main
git push origin main

# Contribute
git switch -c fix/broken-link
git commit -am "Fix broken link in README"
git push -u origin fix/broken-link
gh pr create --repo owner/project
```

Merge Conflicts

Happen when two branches change the **same lines** of the same file.

```
<<<<<<< HEAD
Target IP: 10.10.11.242
=======
Target IP: 10.10.11.250
>>>>>>> feature/update-target
```

1. `git status` — see which files conflict
2. Edit the file: keep the right content, **delete the markers**
3. `git add <file>` — mark resolved
4. `git commit` — finish the merge (or `git merge --abort` to back out)

Avoiding Conflicts & Team Habits

DO

✅ `git pull` before starting work
✅ Small, focused commits & PRs
✅ Clear commit messages (imperative: "Add...", "Fix...")
✅ Communicate who is editing what
✅ Review teammates' PRs

DON'T

❌ Push directly to `main`
❌ `git push --force` on shared branches
❌ Commit secrets, passwords, or keys
❌ Commit huge binaries or build output
❌ "fixed stuff" commit messages

Other GitHub Collaboration Features

Feature	Use
Issues	Track bugs, tasks, and ideas; reference in commits with `#12`
Projects	Kanban-style task boards
Branch protection	Require PR reviews / passing checks before merge to `main`
Actions	Automated CI/CD — run tests or scans on every push
README.md	Front page of the repo — explain what it is and how to use it
Releases & tags	Mark versions: `git tag v1.0 && git push --tags`

🔎 Git as Digital Evidence (IT 360)

WHAT A REPO RECORDS

Every commit stores **author, committer, timestamps, message**, and a SHA hash chaining it to its parent — a built-in change timeline.

```
git log --format="%H %an %ae %ad %s"
```

RECOVERING "DELETED" WORK

`git reflog` shows where HEAD has been; `git fsck --lost-found` finds **dangling** commits after resets or deleted branches.

> **Caution:** author names and dates are set by the client and **can be forged** (`git commit --date`). Corroborate with GitHub server-side data (push events, audit logs) and signed commits. Image the repo before examining it.

📋 Command Cheat Sheet

Setup & Basics	
`git init` / `git clone`	Start
`git status`	Check
`git add`	Stage
`git commit -m`	Save
`git log --oneline`	History
`git diff`	Changes

Branches & Remotes	
`git switch -c`	New branch
`git merge`	Combine
`git push -u origin`	Publish
`git pull` / `git fetch`	Update
`git remote -v`	Remotes
`gh pr create`	Open PR

🧪 Hands-On Lab

1. Create a GitHub account and set up an **SSH key**
2. Create a repo named `it360-git-lab` with a `README.md` and `.gitignore`
3. Clone it, add a file, **commit** and **push**
4. Create a branch `feature/<your-name>`, make a change, push it
5. Open a **pull request**; have a partner **review** and merge it
6. Both partners edit the same line → create and **resolve a conflict**
7. Submit your `git log --oneline --graph --all` output

📚 Resources

- *Pro Git* book (free): git-scm.com/book
- GitHub Docs: docs.github.com
- Interactive branching practice: learngitbranching.js.org
- GitHub Skills courses: skills.github.com
- GitHub CLI: cli.github.com

Questions?

```
git commit -m "Lecture complete"
```